Android FBE Master Key Recovery — Project Summary

Oct 16, 2026

Reproduction of Groß, Busch, Müller (2021), *"One key to rule them all: Recovering the master key from RAM to break Android's file-based encryption,"* built for a Computer Forensics course project.

Overview

Android's File-Based Encryption (FBE) derives a per-file encryption key from a device master key using `AES-ECB` with a **non-secret, on-disk nonce as the encryption key itself**. Because that operation is trivially invertible, anyone who can find a file's key fragment in a RAM dump and read its nonce off the filesystem can recover the master key directly — no brute force required. This is the paper's core finding.

This project reproduces that attack as three independently buildable pieces, mirroring the paper's own structure:

- **`fbekeyrecover`** — the master-key-recovery algorithm itself (Python)
- **`fbe_forensic_toolkit`** — decrypts filenames/content once a master key is known, standing in for the paper's TSK/Plaso patches (Python)
- **`android-fbe-probe`** — an on-device Android app that checks whether a given device is even vulnerable (Kotlin + native C++)

Outcome: the cryptographic core is fully implemented and independently verified correct against synthetic test data. What we could *not* do is reproduce the paper's full positive result on a real device — we traced that to a specific, confirmed environmental limitation (covered below) rather than leaving it unexplained.

The Vulnerability

File-Based Encryption (FBE), introduced in Android 7 and mandatory since Android 10, encrypts each file and folder individually (rather than the whole disk) so that basic phone functions can run before the user unlocks the device.

Every encrypted file has a 16-byte **nonce** (stored, unencrypted, as a filesystem attribute) and is encrypted with a per-file key, `DEKf`, derived from a device **master key** (`MK`) like this:

```
DEKf = AES-128-ECB-Encrypt(key = nonce, data = MK)
```

The nonce — ordinary, non-secret metadata — is used as the *encryption key*, and the master key itself is the data being encrypted. Since AES is a reversible permutation, this equation inverts cleanly:

```
MK = AES-128-ECB-Decrypt(key = nonce, data = DEKf)
```

So: find a file's `DEKf` in a RAM dump, read its nonce off disk, and you can compute the master key that protects *every* file on the device under that key — not just the one file you started from. The paper calls this a flaw in the key derivation function (KDF), since it leans on data that was never meant to be secret.

What We Built

Paper deliverable	Paper section	Our component	Status
`fbekeyrecover` tool	§4.2	`fbekeyrecover/`	Implemented, verified correct (synthetic end-to-end test)
TSK extension (decrypt filenames/content)	§4.1	`fbe_forensic_toolkit/`	Capability reimplemented independently in Python; not a fork of TSK's C codebase
Plaso extension	§4.3	`fbe_forensic_toolkit/plaso_notes.md`	Documented (exact integration point identified), not implemented
Device evaluation app	§5	`android-fbe-probe/`	Implemented, built in Android Studio, run on real emulators

Paper deliverable	Paper section	Our component	Status
EXT4 nonce/descriptor parsing (shared by the above)	§4.1	`fbe_common/`	Implemented, verified correct (hand-built synthetic filesystem test)

Forking and rebuilding The Sleuth Kit and Plaso (two large, separate upstream C/Python projects) wasn't a reasonable scope for a course project and would have mostly tested plumbing rather than the paper's actual cryptographic contribution — so that capability was reimplemented standalone instead, and documented where it is deliberately incomplete.

Implementation Walkthrough: Paper → Code

§2 — Background (key hierarchy). Establishes `DEKf = AES-Encrypt(key=nonce, data=MK)`. This single equation is the entire exploit, and it's the one thing every component ultimately serves. Implemented forward in `fbe_crypto.py::derive_file_key()`, inverted in `master_key_recovery.py::aes128_ecb_decrypt_block()`.

§3.1 — Prerequisites. The paper needs a RAM dump and a copy of the userdata partition, obtained via cold boot / bootloader exploit / chip-off. We don't reproduce acquisition (physical-hardware techniques, out of scope) — our tools consume these as **given input files**, exactly like the paper's own tool does.

§3.2 — The master key derivation attack (the paper's core contribution). Fully implemented in `fbekeyrecover/`: find `DEKf` fragments in RAM via `aeskeyfind`, find nonces on the filesystem via the EXT4 extended attribute, vote on repeating values across every (candidate key, nonce) pairing, concatenate the top two halves. Validated end-to-end with a synthetic test (see Verification, below). The paper's "second KDF" caveat (AES-256-HEH filenames mode → not vulnerable) is detected and explicitly skipped rather than silently mishandled.

§4.1 — TSK extension. Reimplemented as a standalone capability in `fbe_forensic_toolkit/fbe_crypto.py`: AES-256-XTS content decryption, and AES-256-CBC-CTS (ciphertext stealing) filename decryption. The CTS implementation was derived algebraically from the AES-CBC equations directly (documented inline) rather than

copied from a remembered spec — this caught a real bug (a block cipher can't produce ciphertext shorter than one block, so sub-block-length inputs need pre-padding) before it shipped, and the fix was verified with a round-trip self-test across every boundary length.

§4.2 — `fbekeyrecover` tool. `fbekeyrecover/fbekeyrecover.py` ties §3.2's algorithm into a CLI, writing results in the exact `descriptor:part1:part2` format the paper's TSK `-K` argument expects (and which our own `decrypt_tool.py` also reads). While building the `aeskeyfind` wrapper, I initially guessed at its output format from the description alone — then fetched the actual upstream C source to confirm the real format (bare hex lines, no labels unless `-v` is passed) and corrected the parser to match exactly, rather than shipping an untested guess.

§4.3 — Plaso extension. Not implemented, only documented (`plaso_notes.md`) — Plaso's role here is pure plumbing (feed decrypted bytes to its existing, unmodified parsers), and forking three more upstream projects to demonstrate plumbing wasn't worth the scope.

§5 — Evaluation methodology. `android-fbe-probe/` reproduces the paper's own device-vulnerability check: an `ioctl(FS_IOC_GET_ENCRYPTION_POLICY)` call against a device's own data folder. Built as a real Android Studio project and actually run on emulators (not just written) — testing revealed modern Android defaults to a newer fscrypt policy version the original paper didn't need to handle, so a v2 fallback (`FS_IOC_GET_ENCRYPTION_POLICY_EX`) was added as a direct, necessary extension of the paper's own method.

§6/§7 — Conclusion & limitations. The paper's own stated limitation — platform security keeps narrowing this attack surface, a full chain couldn't be run on fully-patched devices — is precisely what we ran into empirically with the probe app (see Real-Device Investigation, below), rather than something we merely read about.

The Voting Concept, In Detail

Reading nonces. Every file/folder in Android's VFS is represented on disk by an **inode** — the structure holding a file's metadata (size, permissions, timestamps, and any **extended attributes**, which are extra key/value metadata beyond the standard fields). Android's fscrypt stores each file's key descriptor (which master key it uses) and nonce as extended attribute index 9 on that file's inode. `fbe_common/ext4_inode_scanner.py` reads this directly from raw inode bytes, without needing to understand directory structure or filenames at all.

Why correct pairings always agree. A master-key half (call it `H`) is one fixed secret, shared by every file encrypted under that master key — it doesn't change per file. What changes per file is the nonce. Think of `H` as a secret sealed inside a box, with a different lock (nonce) used for every file — the box looks different each time, but it always contains the same `H`. Decrypting a file's own `DEKf` fragment with *that file's own* nonce is the exact mathematical inverse of how it was created, so it returns `H` exactly, bit for bit, every time — not approximately, not by coincidence. Six files sharing one master key means up to six separately-recovered confirmations of the identical `H`.

Why wrong pairings are noise. Decrypting with the *wrong* nonce doesn't "almost" work — AES produces pseudorandom, unrelated output. Since `aeskeyfind` returns an unlabeled pool of every AES-looking key fragment anywhere in the RAM dump (no idea which file, or even which master key, any of them belongs to), the algorithm tries *every* candidate against *every* nonce in one descriptor bin. The handful of correct pairings all land on the identical value `H`; the thousands of wrong pairings scatter across the 2^{256}-value space, with a collision between two different wrong answers being astronomically unlikely.

Why it tolerates memory loss. Tally every decryption result (`Counter` in `recover_master_key_halves()`); the value with the most hits is overwhelmingly likely to be real. Only **two** surviving file keys from the same master key are needed to produce a detectable repeat against the single-hit noise floor — which is exactly why the paper calls this method robust to the partial bit-rot a cold boot attack causes.

Module Reference

`fbe_common/`

- `ext4_inode_scanner.py` — minimal, dependency-free EXT4 reader. Parses the superblock, locates the inode table, and reads the fscrypt extended attribute (descriptor, nonce, encryption modes) off any given inode. Does not implement directory traversal or content extraction — only what the key-recovery attack needs.

`fbekeyrecover/`

- `aeskeyfind_runner.py` — subprocess wrapper around the external `aeskeyfind` tool; parses its real stdout format (confirmed against upstream source).
- `master_key_recovery.py` — the voting algorithm: pairs every candidate key against every nonce in a descriptor bin, tallies results, returns the top-voted master-key halves.
- `fbekeyrecover.py` — CLI entry point tying the above together into the full attack.

- `selftest.py` — synthetic end-to-end validation of the whole chain (see Verification, below).

`fbe_forensic_toolkit/`

- `fbe_crypto.py` — `derive_file_key()` (forward KDF), `decrypt_file_content()` (AES-256-XTS), `cbc_cts_encrypt` / `decrypt` (hand-derived ciphertext stealing), `decrypt_filename()`.
- `decrypt_tool.py` — CLI: load recovered master keys, look up a file's encryption context, decrypt its filename or content.
- `plaso_notes.md` — documents the real Plaso/dfVFS integration point without implementing it.

`android-fbe-probe/`

- `fbe_probe.cpp` (native/NDK) — opens the app's own data folder, tries the legacy `FS_IOC_GET_ENCRYPTION_POLICY` ioctl, falls back to the newer `..._EX` (v2) ioctl if that fails.
- `EncryptionPolicyResult.kt` — data class mirroring the native result (errno, policy version, modes, key bytes).
- `FscryptModes.kt` — named encryption-mode constants and the actual vulnerability check from §3.2/§7.
- `NativeProbe.kt` — thin JNI binding.
- `MainActivity.kt` — UI logic: runs the probe, interprets and displays the result, also shells out to `mount` mirroring the paper's own evaluation app.

Verification & Testing

Three self-tests were actually written and run — not just claimed:

1. **CBC-CTS round-trip self-test** (`fbe_crypto.py`) — encrypts and decrypts random data across every boundary length (exact block multiples, one byte over/under, single block, multi-block), confirming the hand-derived ciphertext-stealing math is internally consistent. This is also where a real bug was caught: the original implementation tried to handle inputs shorter than one AES block, which is mathematically impossible for a block cipher to do invertibly — fixed before shipping.
2. **Synthetic EXT4 byte-level parser test** (`fbe_common/test_ext4_inode_scanner.py`) — hand-builds a fake EXT4 image (superblock, one group descriptor, one inode with an inline extended attribute) as raw bytes, and asserts the scanner parses it back exactly.

This caught a real bug: the group-descriptor-table location formula was off by one block for 1024-byte-block filesystems.

3. **Full synthetic end-to-end attack chain** (`fbekeyrecover/selftest.py`) — generates a real random 512-bit master key and six nonces, derives the real per-file keys the kernel would produce, mixes them into 500 random noise keys (simulating an entire RAM dump's worth of unrelated AES-looking data), and confirms the voting algorithm recovers the exact original master key. This is independent proof the cryptographic core works, without needing any real device, RAM dump, or disk image.

All three pass. Together they validate the EXT4 parsing, the cryptography, and the attack algorithm independently of each other and independently of any real forensic input — which matters because, as covered below, no real memory dump or extracted device image was ever available to test against.

Real-Device Investigation

The `android-fbe-probe` app was built and actually run on real Android emulators (not just written): an AVD on Android 10 (API 29, the oldest config with FBE at all per the paper) and one on Android 16 (API 36), both using Google APIs system images rather than Google Play Store images — deliberately, since Play Store images are production ("user") builds that lock out root and SELinux toggling, while Google APIs images are "userdebug" builds that allow both.

Three causes were tested and ruled out one at a time, rather than guessed at once and left unverified:

1. **SELinux.** Enforcing-mode logcat showed an identical `avc: denied { ioctl }` entry on *both* API levels, decoding to exactly the right ioctl (`FS_IOC_GET_ENCRYPTION_POLICY`). An earlier hypothesis that this was recent, version-specific Android hardening turned out to be wrong once API 29 showed the identical denial — corrected rather than left standing. Bypassed with `adb root` + `adb shell setenforce 0`, confirmed via `permissive=1` appearing in the subsequent logcat entries (still logged, no longer blocking).
2. **fscrypt policy version.** With SELinux confirmed bypassed, the ioctl still failed (`ENOTTY`) — a kernel-level answer, not a permission wall. Suspecting a v1-vs-v2 fscrypt policy mismatch (modern Android defaults to v2), a v2 fallback (`FS_IOC_GET_ENCRYPTION_POLICY_EX`) was added to the native probe. It *also* failed with `ENOTTY` — ruling this out too.

3. **Root cause.** The mount table showed `/data/user/0` sitting on a device-mapper block (`dm-52`) — genuine whole-volume/metadata-level encryption is active, matching the paper's own description of "FDE used on top of FBE" on recent devices. But the specific *per-app* fscrypt policy the paper's attack targets was never individually provisioned on that directory, despite `ro.crypto.type=file` reporting encryption as active system-wide.

Conclusion: generic AOSP emulator images don't appear to provision real per-app FBE policies, even when whole-volume encryption is genuinely active. The paper's own evaluation (Table 1/2) used exclusively physical OEM hardware (Nexus, Pixel, Samsung, Huawei, Xiaomi) — never emulators — which fits what we found. A rooted physical Android device remains the credible next step for a genuinely positive result.

What's Implemented vs. Not

Implemented and independently verified correct: the master-key-recovery algorithm (§3.2), the EXT4 extended-attribute parser (§4.1's data layer), and the filename/content decryption cryptography (§4.1's TSK capability) — all three validated against synthetic test data with real, run self-tests, not just written and assumed correct.

Implemented, built, and run (but without the paper's expected positive result): the on-device evaluation app (§5) — the code is verified correct (confirmed via logcat that the exact right ioctl reaches the kernel every time), but it never once returned the paper's "vulnerable, old KDF" result on any emulator tested, for the environmental reasons detailed above.

Deliberately out of scope: forking The Sleuth Kit or Plaso (§4.1/§4.3) — the equivalent cryptographic capability was reimplemented standalone instead, with the real integration points documented for anyone who wants to finish the plumbing.

Not reproduced: the paper's actual end-to-end positive result — a real device where the attack succeeds against genuine RAM-dump and disk-image data. No real memory dump or extracted userdata partition was ever acquired or available; all Python-side testing used synthetic data constructed specifically to validate the code in isolation.

Next step: testing against a rooted physical Android device would be the realistic way to close this gap — both to get a true positive from the probe app, and to finally exercise `fbekeyrecover` / `fbe_forensic_toolkit` against real forensic input instead of synthetic data.